

Enterprise Banking Microservices

A scalable and secure distributed banking backend — course project for Database Systems Engineering and Distributed Backend Development.\

Team:

Snigdha 2520090160

Koumudi 2520090161

Sumanjali 2520090190

Problem Statement

Traditional banking systems struggle with growth, security, and operational complexity. Key challenges:

- Scaling for high transaction volumes
- Secure authentication and authorization
- Consistent, reliable transaction processing
- Managing payments and account state across services
- Detecting suspicious/fraudulent behavior in real time



Project goal: build a secure, modular microservices platform managing customers, accounts, transactions, and fraud detection.

Project Objectives

Secure Architecture

Implement authentication, authorization, and secure REST APIs.

Scalability

Microservices design to scale individual capabilities independently.

Transaction Integrity

Consistent account balances and reliable transaction logging.

Fraud Detection

Identify suspicious patterns and flag high-risk activities.

User Experience

Clean frontend: registration, dashboard, transfers, and forms.

Literature Survey — Summary

Reviewed domains and high-level findings. (Final slide will list exact papers and citations.)



Microservices Architecture

Improves modularity and scale; introduces distributed communication and consistency challenges.



Secure Authentication

Token-based auth and role-based access control are effective but need careful token lifecycle management.



Banking Transaction Systems

ACID semantics support correctness; scaling requires careful partitioning and compensating transactions.



Fraud Detection

Pattern-based and ML approaches detect anomalies; accuracy depends on data richness and labeled examples.



Research gap: Few works integrate scalable microservices, strong transactional guarantees, and inline fraud detection in a single platform.

Proposed System — Overview

We propose a microservices-based banking platform exposing secure REST APIs. Core capabilities:

- Customer management and registration
- Authentication & authorization (token-based)
- Account and balance management
- Secure money transfers and payment recording
- Transaction ledger and audit trails
- Real-time fraud detection and alerting

Technology stack: React.js (frontend) → FastAPI (microservices) → MySQL (persistent store)

System Architecture — Flow

Frontend (React) handles UI and client validation. FastAPI services implement business logic per bounded context (auth, accounts, payments, fraud). Services communicate over REST (HTTP) and use MySQL for persistent state. Fraud service subscribes to transaction events for near-real-time scoring.



Modules & Responsibilities



Customer Management

Profile CRUD, KYC fields, linkage to user accounts.



Authentication & Authorization

JWT tokens, refresh tokens, RBAC for operations.



Account Management

Account types, balance updates, overdraft rules.



Transfer & Payments

Atomic transfer flows with idempotency and audit logs.



Transaction Management

Durable transaction ledger and reconciliation endpoints.



Fraud Detection

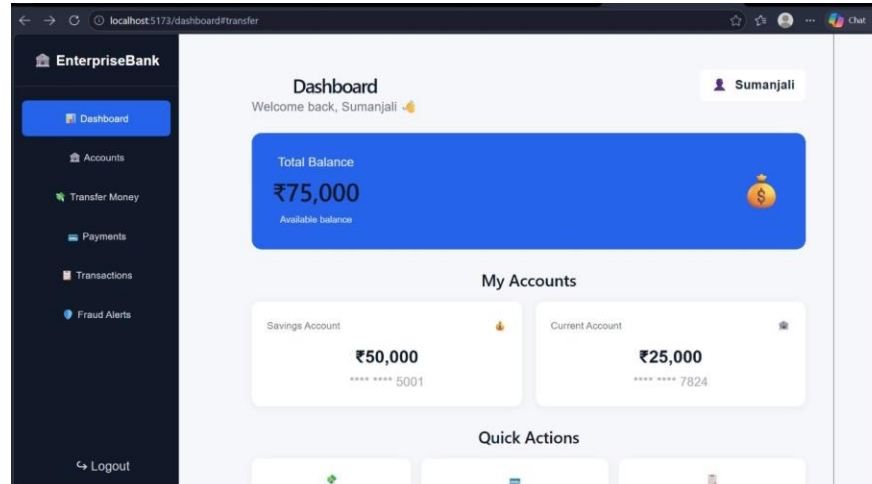
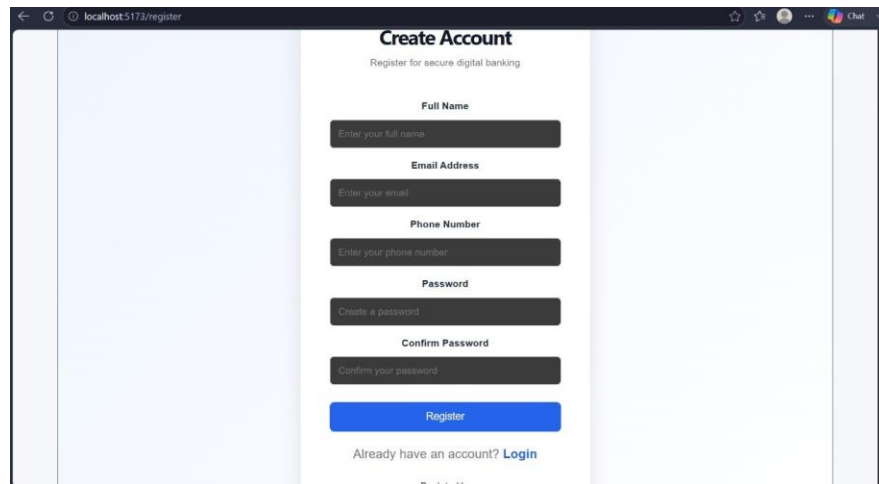
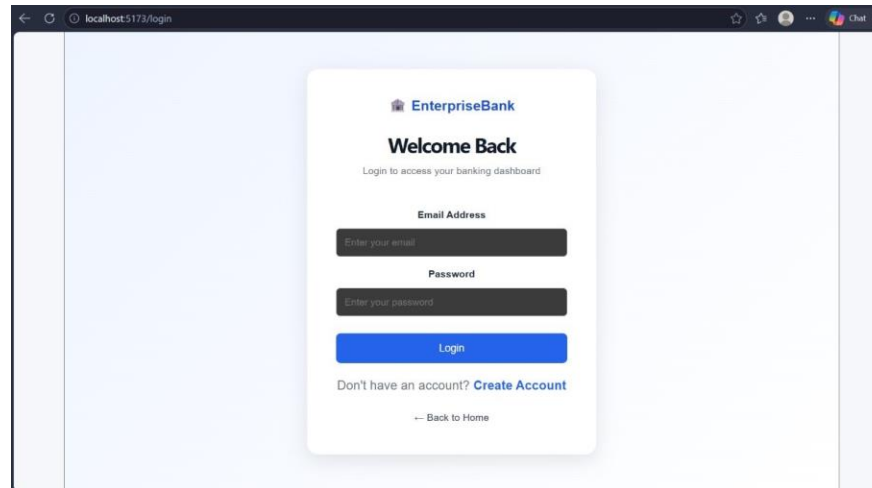
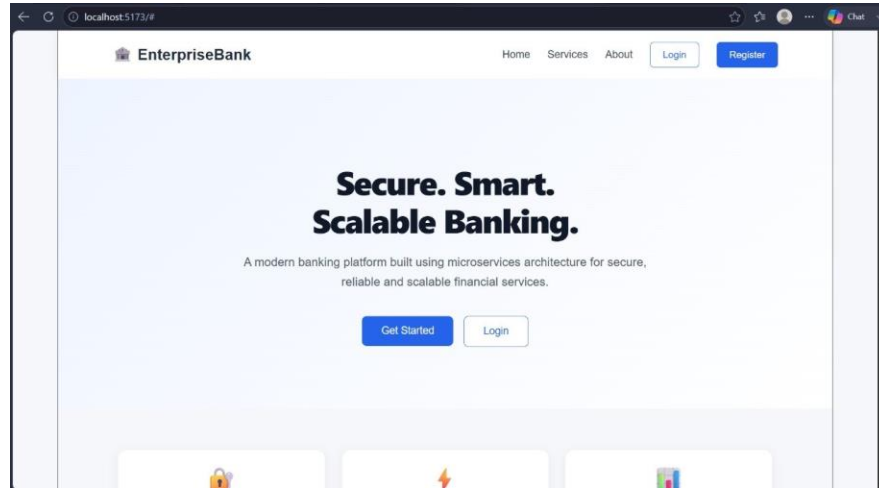
Rule-based scoring and anomaly flags; alerts + quarantine actions.



Banking Dashboard

Aggregate views: balances, recent activity, alerts, KPIs.

Frontend Design (Simple & Human)



Frontend Design (Simple & Human)

← Back to Dashboard



Transfer Money

Send money securely to another account

From Account

Recipient Account Number

Enter recipient account number

Transfer Amount

₹ Enter amount

Description

Enter transaction description

Continue Transfer



Fraud Risk Analysis

← Back to Dashboard



Fraud Detection

Monitor transactions and identify suspicious activity.

Transactions Analyzed

128

Suspicious Transactions

7

High Risk

2

Average Risk Score

18/100

Recent Risk Analysis

Transaction	Amount	Risk Score	Status
TXN001	₹5,000	12/100	LOW
TXN002	₹45,000	78/100	HIGH
TXN003	₹2,500	21/100	LOW
TXN004	₹30,000	65/100	REVIEW

Database Design — Conceptual

MySQL relational schema — main tables and key relationships:

- customers (customer_id, name, KYC, contact_info)
- users (user_id, customer_id, username, password_hash, roles)
- accounts (account_id, customer_id, balance, currency, status)
- transactions (tx_id, from_account, to_account, amount, status, timestamp)
- payments (payment_id, account_id, payee, amount, method, status)

Relationships: customers→users (1:N), customers→accounts (1:N), accounts→transactions (1:N). Use indexes on account_id, customer_id, and transaction timestamps for performance.



Next Steps & Implementation Plan

01

Phase 1 — Core Services

Implement auth, customer, accounts, and transaction services with MySQL persistence and API contracts.

03

Phase 3 — Fraud Detection

Integrate rule-engine, event stream ingestion, and alerting; evaluate ML models with labeled data.

02

Phase 2 — Transfers & Idempotency

Design atomic transfer flows, idempotency keys, and reconciliation endpoints.

04

Phase 4 — Frontend & Testing

Polish UI, add screenshots (presenter to insert), run integration and load tests; prepare final paper with citations.

Deliverables: working microservices, API documentation, dataset for fraud evaluation, final report with literature citations.



Enterprise Banking Microservices — Architecture & Implementation

Concise technical overview for graduate-level reviewers: schema, backend design, auth, transfers, fraud detection, and next steps. Theme accents: #3E2513 and #F3E3D8.



Agenda

Data Model

ER diagram & key schema tables

Sample Data

Representative MySQL rows and
snapshot

Backend & Security

FastAPI, JWT, bcrypt, protected
endpoints

Transfers & Fraud

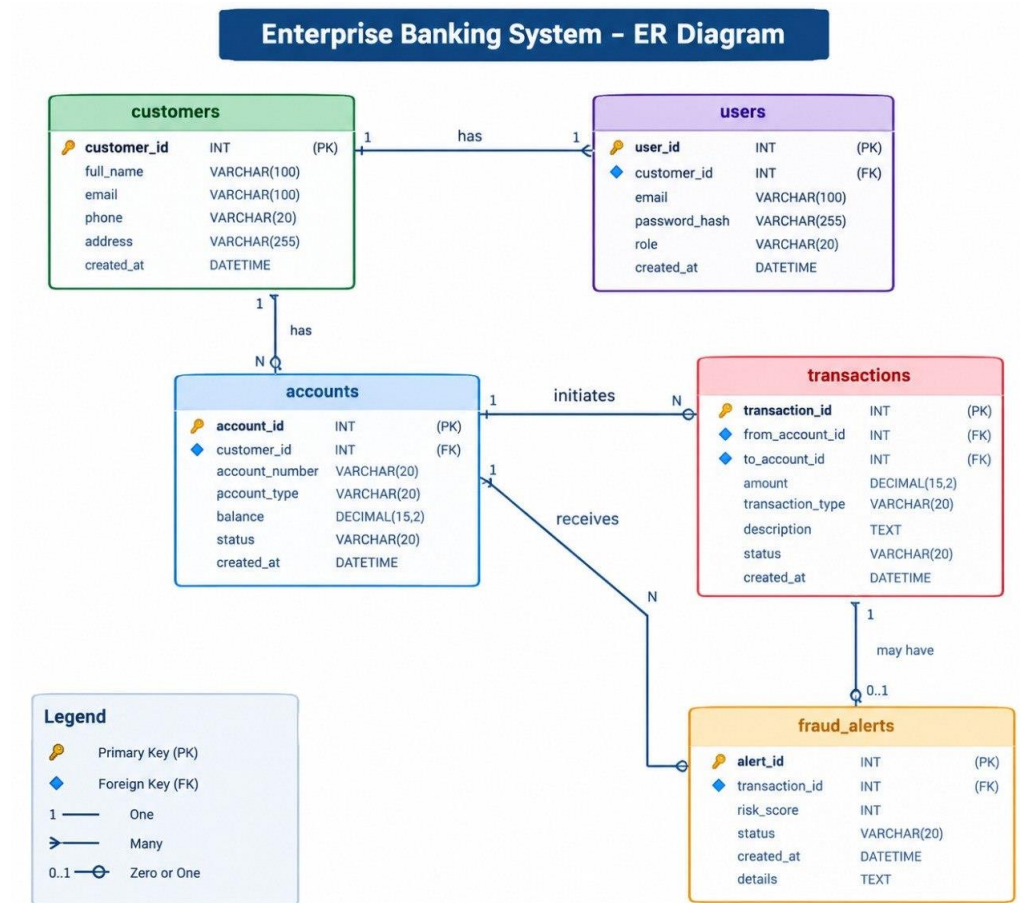
Flow, atomicity, DBSCAN anomaly detection

Challenges & Roadmap

Integration, deployment, advanced alerts

ER Diagram — Core Entities

Visual: customers, users, accounts, transactions, payments with PK/FK links (customers ← users, customers ← accounts, accounts ← transactions, transactions ← payments). See diagram for primary/foreign-key relationships and cardinality.



Schema — Key Tables (selected columns)

Customers

- customer_id (PK)
- full_name
- email
- phone
- address
- created_at

Accounts

- account_id (PK)
- customer_id (FK → customers.customer_id)
- account_number
- account_type
- balance
- status

Users

- user_id (PK)
- customer_id (FK → customers.customer_id)
- email
- password_hash
- role
- created_at

Transactions / Payments

transaction_id (PK), account_from_id (FK), account_to_id (FK),
amount, currency, timestamp, status, payment_id (FK optional)

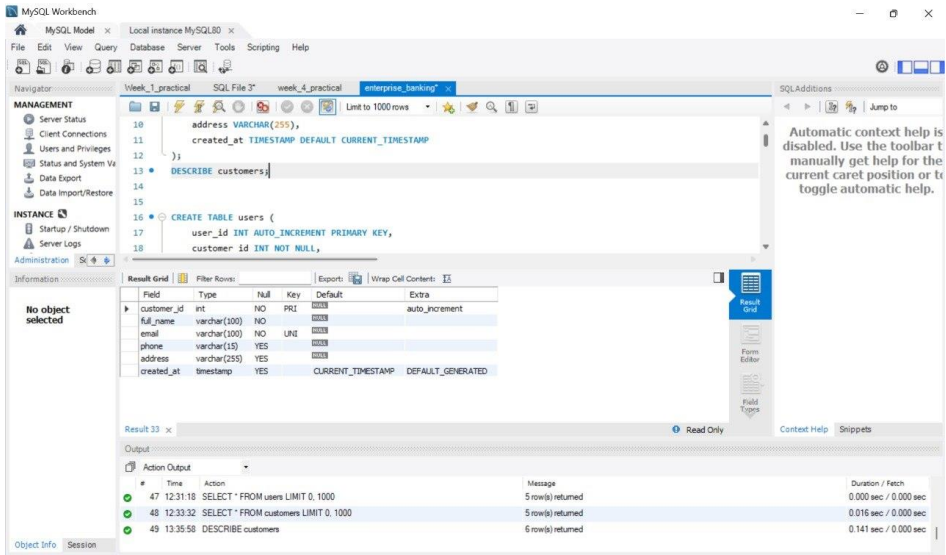


Sample Records — Accounts (MySQL)

Representative rows pulled directly from MySQL (formatted):

account_id	customer_id	account_no	type	balance
1	1500	SAVINGS	115001SAVINGS	₹50,000
2	17824	CURRENT	217824CURRENT	₹25,000
3	26001	SAVINGS	326001SAVINGS	₹40,000
4	37001	SAVINGS	437001SAVINGS	₹30,000

Statuses: ACTIVE for all shown. These rows are exact MySQL excerpts used for demonstrations and testing.



Small MySQL output snapshot (for auditor verification).

Backend Design — FastAPI Overview



REST APIs

Customer registration, user login, accounts, transactions, dashboard endpoints.



Database Layer

MySQL with connection pooling; transactions use REPEATABLE READ / explicit FOR UPDATE where needed.



Business Logic

Transfer orchestration, balance checks, idempotency keys for retry safety.

Authentication & Security

Flow: User → submit credentials → backend verifies bcrypt hash → generate JWT (exp, sub, roles) → client stores token → token used for protected endpoints.

- Bcrypt for password hashing and salting
- JWT with short expiry + refresh token pattern
- Role-based authorization checks at endpoint level
- HTTPS-only, secure cookies or Authorization header

📌 Frontend-backend integration is incomplete; do not claim full end-to-end UI connectivity.

Money Transfer — Flow & Example



Authenticate User

Verify Ownership

Check Balance

Execute Transaction

Example: Transfer ₹5,000 from Account 1 (₹50,000) to Account 3 (₹40,000) → Result: Account 1 = ₹45,000, Account 3 = ₹45,000. Use DB transactions with SELECT ... FOR UPDATE and idempotency keys to prevent double-spend.

Fraud Detection — Approach

Goal: surface anomalous transactions with low false positives while enabling automated flagging for human review.

- Feature set: amount z-score, velocity (tx/min), geo deviation, device fingerprint, payee history
- Model: unsupervised DBSCAN for clustering typical behavior; outliers → risk scores
- Pipeline: batch score (nightly) + streaming alerts for high-risk tx

 DBSCAN is chosen to detect density-based outliers without labeled fraud data; tune eps/min_samples and normalize features.

Challenges, Roadmap & Conclusion

Key Challenges

Maintaining transactional integrity, secure credential storage, safe retry semantics, and robust fraud signal integration.

Next Steps

Complete React-FastAPI integration, wire fraud model into transaction API, add payment provider integration, containerize with Docker and deploy to cloud.

Conclusion

Modular stack (React + FastAPI + MySQL + JWT) provides a secure, extensible foundation for banking features and analytic extensions. Prioritize safe transfer semantics and fraud integration before production rollout.

